

USEVALAD MILASHEUSKI/10816982

1) (coap.mid == 53533 || coap.mid == 42804) && not coap.type == 2

Using filter coap.mid == MID, we can see that MID 53533 corresponds to a confirmable REQ message + an acknowledgement which we filter out by using not coap.type == 2, while MID 42804 corresponds to a non-confirmable one.

Main differences:

- MID 53533 – a) Code GET; b) Token 242f92f0; c) Confirmable; d) Source port 50593
- MID 42804 – a) Code DELETE; b) Token 6dbdd020; c) Non-confirmable; d) Source port 5683

No.	Time	Source	Destination	Protocol	Length	Info
6276	126.7948172...	10.0.2.15	194.196.15.150	CoAP	60	NON, MID:42804, DELETE, TKN:6d bd d0 20, End of Block #0, /link2
6294	126.9412843...	10.0.2.15	134.102.218.18	CoAP	61	CON, MID:53533, GET, TKN:24 2f 92 f0, Block #5, /large

2) coap.response_to == 2428

Firstly we apply frame.number == 2428 to check the request. It is a delete request, which is a non-confirmable, but since it is a DEL request, the user is supposed to receive a response. Indeed, if we apply) coap.response_to == 2428 we will see that it is true, the message contains code 2.02 (successfully deleted) and the tokens of the two messages coincide(coap.token == 67:c7:22:9a. will output two messages: from the user and from the server).

No.	Time	Source	Destination	Protocol	Length	Info
2429	46.613975481	127.0.0.1	127.0.0.1	CoAP	72	NON, MID:844, 2.02 Deleted, TKN:67 c7 22 9a, End of Block #0, /living_room/door

3) coap.code == 69 && ip.dst == 127.0.0.1 -> 8 replies

Since the IP of the localhost is 127.0.0.1 and the code corresponding to content is 69, by combining the two filters we obtain eight received ACKs to confirmable messages.

No.	Time	Source	Destination	Protocol	Length	Info
90	9.704166705	127.0.0.1	127.0.0.1	CoAP	69	ACK, MID:63229, 2.05 Content, TKN:11 fd c7 72, End of Block #0, /living_room
1047	23.625323733	127.0.0.1	127.0.0.1	CoAP	69	ACK, MID:4920, 2.05 Content, TKN:b6 3d cc 6d, End of Block #0, /living_room
1337	36.639686319	127.0.0.1	127.0.0.1	CoAP	69	ACK, MID:23246, 2.05 Content, TKN:e8 cf 79 21, End of Block #0, /living_room
2124	41.646753209	127.0.0.1	127.0.0.1	CoAP	69	ACK, MID:13240, 2.05 Content, TKN:47 a9 45 65, End of Block #0, /living_room
2537	52.663764603	127.0.0.1	127.0.0.1	CoAP	69	ACK, MID:29961, 2.05 Content, TKN:6d cf 30 dd, End of Block #0, /living_room
2673	58.668864968	127.0.0.1	127.0.0.1	CoAP	69	ACK, MID:25273, 2.05 Content, TKN:b6 69 52 f1, End of Block #0, /living_room
2921	81.689542540	127.0.0.1	127.0.0.1	CoAP	69	ACK, MID:48882, 2.05 Content, TKN:b5 cc 8e 5b, End of Block #0, /living_room
3055	90.691363524	127.0.0.1	127.0.0.1	CoAP	69	ACK, MID:21099, 2.05 Content, TKN:f8 eb 78 0b, End of Block #0, /living_room

4) !coap.opt.observe && (coap.code == 132) -> 8 requests

Excluding the observable message at first, we apply the filter which corresponds to a 4.04 Not Found code, so we apply 132 code and observe 8 messages, 6 of which are ACKs and 2 of which are NON. We assume that number of requests equal to number of the replies (none was lost).

2431	46.618581580	127.0.0.1	127.0.0.1	CoAP	52	ACK, MID:15597, 4.04 Not Found, TKN:ff 48 81 30
2713	61.615996450	127.0.0.1	127.0.0.1	CoAP	52	ACK, MID:10267, 4.04 Not Found, TKN:3a 3e 3f e4
2769	69.630046959	127.0.0.1	127.0.0.1	CoAP	52	NON, MID:847, 4.04 Not Found, TKN:cc a0 32 da, /living_room/door
2807	71.643549576	127.0.0.1	127.0.0.1	CoAP	52	ACK, MID:54030, 4.04 Not Found, TKN:ec 83 12 6a
2837	74.651922742	127.0.0.1	127.0.0.1	CoAP	52	ACK, MID:2441, 4.04 Not Found, TKN:3c 67 cb 2b
3053	90.647929780	127.0.0.1	127.0.0.1	CoAP	52	ACK, MID:57705, 4.04 Not Found, TKN:ae af 66 d6
4912	113.6753035...	127.0.0.1	127.0.0.1	CoAP	52	ACK, MID:27097, 4.04 Not Found, TKN:88 db 4a 0b
5562	120.6919564...	127.0.0.1	127.0.0.1	CoAP	52	NON, MID:158, 4.04 Not Found, TKN:a5 9e dc 37, /living_room/door

5) tcp.srcport in {51565 41869 60395 40989 47135 44429 60419 55953} && ip.src == 127.0.0.1 && mqtt.topic matches "factory/departement./+" && mqtt.msgtype == 3 -> 13 messages

5797	122.3929255...	127.0.0.1	127.0.0.1	MQTT	96	Connect Command
4409	109.3011719...	127.0.0.1	127.0.0.1	MQTT	151	Connect Command
3524	106.6712842...	127.0.0.1	127.0.0.1	MQTT	94	Connect Command
1651	39.405165694	127.0.0.1	127.0.0.1	MQTT	94	Connect Command
1623	39.400986803	127.0.0.1	127.0.0.1	MQTT	119	Connect Command
1515	39.383836950	127.0.0.1	127.0.0.1	MQTT	169	Connect Command
322	21.128297542	127.0.0.1	127.0.0.1	MQTT	95	Connect Command
163	21.083444218	127.0.0.1	127.0.0.1	MQTT	128	Connect Command

Firstly we use mqtt.passwd == admin to check the users having the password "admin".

177	21.083974458	127.0.0.1	127.0.0.1	MQTT	211 Publish Message (id=3)	[factory/department1/section2/hydraulic_valve]
412	21.146691589	127.0.0.1	127.0.0.1	MQTT	201 Publish Message (id=1)	[factory/department2/section3/deposit]
1570	39.395950925	127.0.0.1	127.0.0.1	MQTT	199 Publish Message (id=3)	[factory/department1/section1/plc]
1629	39.401658935	127.0.0.1	127.0.0.1	MQTT	210 Publish Message (id=1)	[factory/department2/section1/hydraulic_valve]
1668	39.405621359	127.0.0.1	127.0.0.1	MQTT	200 Publish Message (id=1)	[factory/department1/section4/deposit]
2540	53.418595851	127.0.0.1	127.0.0.1	MQTT	213 Publish Message (id=2)	[factory/department2/section1/hydraulic_valve]
2794	70.096434764	127.0.0.1	127.0.0.1	MQTT	210 Publish Message (id=3)	[factory/department1/section2/hydraulic_valve]
3132	98.453437832	127.0.0.1	127.0.0.1	MQTT	207 Publish Message (id=3)	[factory/department2/section1/hydraulic_valve]
3535	106.671672168	127.0.0.1	127.0.0.1	MQTT	214 Publish Message (id=2)	[factory/department2/section2/hydraulic_valve]
4420	109.301635672	127.0.0.1	127.0.0.1	MQTT	202 Publish Message (id=1)	[factory/department2/section2/deposit]
5219	114.454697590	127.0.0.1	127.0.0.1	MQTT	194 Publish Message (id=4)	[factory/department1/section1/plc]
5803	122.393211388	127.0.0.1	127.0.0.1	MQTT	201 Publish Message (id=1)	[factory/department2/section1/plc]
6239	125.318391130	127.0.0.1	127.0.0.1	MQTT	200 Publish Message (id=2)	[factory/department2/section1/deposit]

Than to each of the users we check the TCP source and IP in order to track only the needed users. Since we are interested in the publications, we specify it with `mqtt.msgtype == 3`. For this solution it is assumed that we are looking for any hierarchy lower than `department*/`, where `*` is a number. By using `mqtt.topic matches "factory/department./+"` we specify the lower hierarchy which is similar to the one of interest.

6) in the terminal: ping test.mosquitto.org

`mqtt.conflag.willflag == 1 && ip.dst == 5.196.95.208 -> 9 clients`

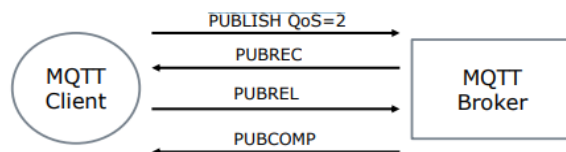
PING test.mosquitto.org (5.196.95.208) 56(84) bytes of data.

No.	Time	Source	Destination	Protocol	Length	Info
693	21.199355426	10.0.2.15	5.196.95.208	MQTT	133	Connect Command
738	21.202674757	10.0.2.15	5.196.95.208	MQTT	125	Connect Command
964	21.367571559	10.0.2.15	5.196.95.208	MQTT	149	Connect Command
1895	39.439393322	10.0.2.15	5.196.95.208	MQTT	149	Connect Command
3903	106.7253628...	10.0.2.15	5.196.95.208	MQTT	157	Connect Command
4659	109.3316829...	10.0.2.15	5.196.95.208	MQTT	157	Connect Command
4725	109.3459837...	10.0.2.15	5.196.95.208	MQTT	145	Connect Command
5393	119.3760713...	10.0.2.15	5.196.95.208	MQTT	133	Connect Command
6551	127.4740446...	10.0.2.15	5.196.95.208	MQTT	133	Connect Command

First in the comand window we *ping* mosquitto broker in order to obtain the IP. Then we use the adress as the destination and search for messages containing a last will message(checking via the flag) and observe that there are nine packages. Since the last will can be specified only in a connect message, specefication of the code is unnecessary.

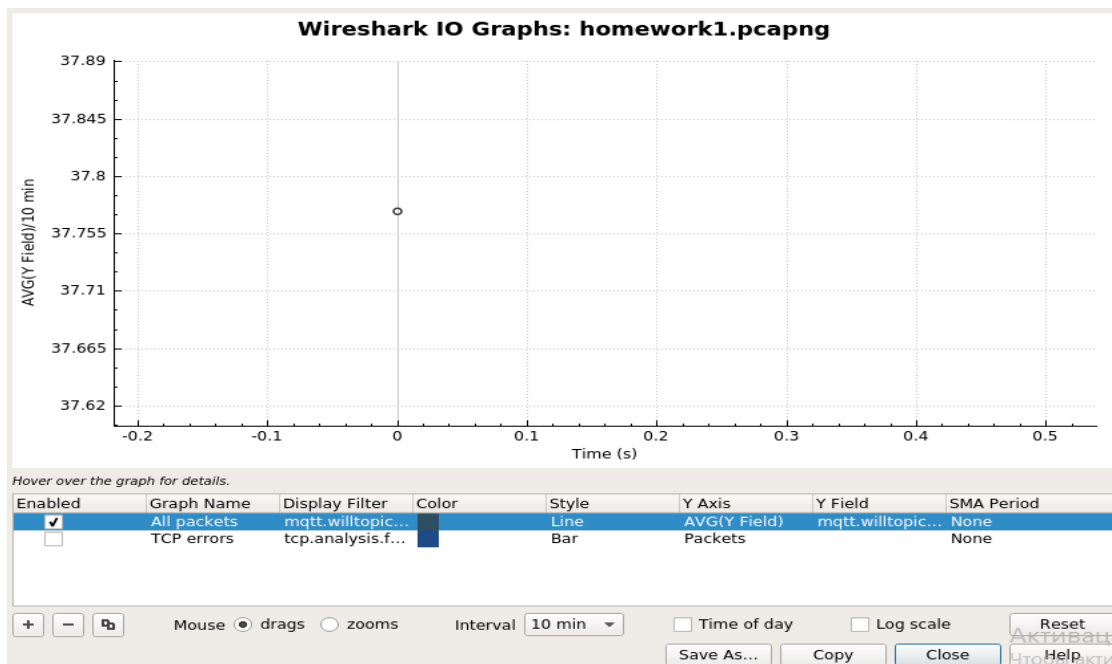
7) `mqtt.msgtype == 6 || (mqtt.qos ==2 && mqtt.msgtype == 3) -> 95 publishes/94 frames`

No.	Time	Source	Destination	Protocol	Length	Info
232	21.102993264	10.0.2.15	18.185.199.22	MQTT	199	Publish Message (id=1) [factory/department1/section1/hydraulic_valve]
6021	122.4194844...	10.0.2.15	3.120.68.56	MQTT	189	Publish Message (id=1) [factory/department1/section1/plc]
1665	39.405598382	10.0.2.15	3.120.68.56	MQTT	192	Publish Message (id=1) [factory/department1/section2/deposit]
4484	109.3010870...	127.0.0.1	127.0.0.1	MQTT	209	Publish Message (id=1) [factory/department1/section2/hydraulic_valve]
2058	39.508634822	10.0.2.15	137.135.83.217	MQTT	187	Publish Message (id=1) [factory/department1/section2/plc]
1544	39.392471332	10.0.2.15	18.185.199.22	MQTT	188	Publish Message (id=1) [factory/department1/section3/deposit]
3402	106.6611216...	127.0.0.1	127.0.0.1	MQTT	200	Publish Message (id=1) [factory/department1/section3/deposit]
5469	119.4489490...	10.0.2.15	137.135.83.217	MQTT	199	Publish Message (id=1) [factory/department1/section4/hydraulic_valve]
282	21.119311127	127.0.0.1	127.0.0.1	MQTT	200	Publish Message (id=1) [factory/department1/section4/plc]
4420	109.3016356...	127.0.0.1	127.0.0.1	MQTT	202	Publish Message (id=1) [factory/department2/section1/deposit]
4712	109.3414798...	10.0.2.15	3.120.68.56	MQTT	201	Publish Message (id=1) [factory/department2/section1/hydraulic_valve]
776	21.214604685	10.0.2.15	3.120.68.56	MQTT	193	Publish Message (id=1) [factory/department2/section2/deposit]
3160	100.2133827...	127.0.0.1	127.0.0.1	MQTT	205	Publish Message (id=1) [factory/department2/section3/deposit]
948	21.253598720	10.0.2.15	137.135.83.217	MQTT	196	Publish Message (id=1) [factory/department2/section3/hydraulic_valve]
5843	122.3978951...	127.0.0.1	127.0.0.1	MQTT	208	Publish Message (id=1) [factory/department2/section4/hydraulic_valve]
4039	106.8148941...	10.0.2.15	137.135.83.217	MQTT	188	Publish Message (id=1) [factory/department3/section1/deposit]
1110	26.179005395	10.0.2.15	5.196.95.208	MQTT	184	Publish Message (id=1) [factory/department3/section1/plc]
6521	127.4590087...	10.0.2.15	3.120.68.56	MQTT	189	Publish Message (id=1) [factory/department3/section2/plc]
2514	49.572622744	10.0.2.15	137.135.83.217	MQTT	198	Publish Message (id=1) [factory/department3/section4/hydraulic_valve]
6665	127.5632584...	10.0.2.15	137.135.83.217	MQTT	195	Publish Message (id=1) [factory/department3/section4/hydraulic_valve]



With the reference to the diagram above, we see that PUBREL is a message the client is supposed to sends to the broker after receiving PUBREC for every publish message of the QoS 2. However, if we check the number of PUBRELS using `mqtt.msgtype == 6`, we see none. So, for this solution it is assumed that thus none of the QoS 2 publishes has received it and, thus the number of messages which have not received PUBREL is equal to the number of QoS 2 PUBLISHes. To count the frames we can use Statistics->Capture Filter Properties and see the number of displayed packages 94, but it is important to notice, that frame 1152 has two PUBLISHes, leading to the final answer 95.

8) `mqtt.willtopic_len && !_ws.malformed && mqtt.clientid==" -> 37.76`



Firstly, if we apply the `mqtt.willtopic_len` filter in the wireshark we can see that there are any malformed packages. In order to avoid them `!_ws.malformed` filter was applied. Then since we care only about empty client IDs we use `mqtt.clientid==""` filter. In order to find the average length, we went to Statistics->I/O Graph. There we inserted the final filter in the "Display Filter" field, changed the Y axis to AVG, inserted `mqtt.willtopic_len` in the Y Field, increased the interval to capture everything and observed that the average is approximately equal to 37.76.

9) `tcp.dstport == 46295 && ip.dst == 10.0.2.15 && (mqtt.msgtype == 2 || mqtt.msgtype == 4 || mqtt.msgtype == 9 || mqtt.msgtype == 11)->5 ACKs`

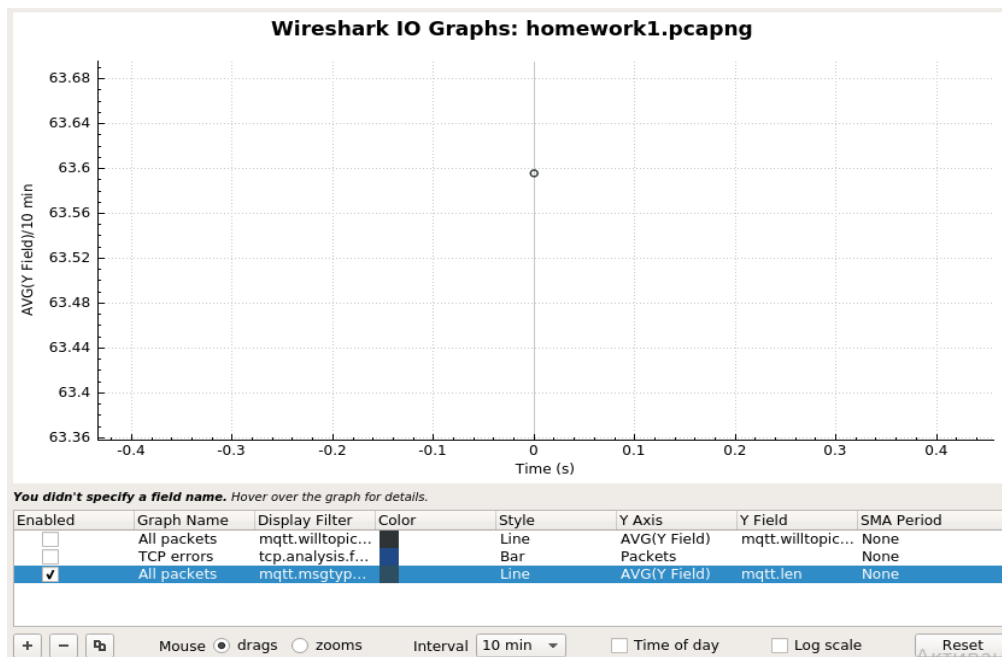
No.	Time	Source	Destination	Protocol	Length	Info
4659	199.3316829...	10.0.2.15	5.196.95.208	MQTT	157	Connect Command

At the beginning we apply `mqtt.clientid == 6M5H8y3HJD5h4EEscWknTD` to discover the message with the given ID.

No.	Time	Source	Destination	Protocol	Length	Info
4764	199.3581143...	5.196.95.208	10.0.2.15	MQTT	62	Connect Ack
4768	199.3592181...	5.196.95.208	10.0.2.15	MQTT	192	Publish Message [factory/department2/section3/deposit], Publish Ack (id=4)
4766	199.3585525...	5.196.95.208	10.0.2.15	MQTT	207	Subscribe Ack (id=1), Publish Message [factory/department1/section4/deposit], Subscrib...

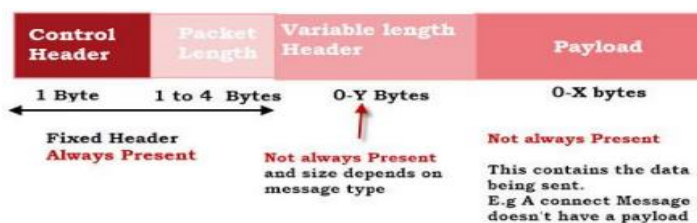
Then we apply filters specifying destination port and destination ip of the use client. Then we add only the acknowledgment codes (2, 4, 9 and 11). As a result, we can observe 3 frames. If we open the packets and count the ACKs, in total we have a Connection acknowledgment in frame 4764, Publish acknowledgment in frame 4768 and 3 Subscribe acknowledgments in frame 4766, which leads to 5 in total.

10) `mqtt.ver == 3 && mqtt.msgtype == 1->63.59`



Filter corresponding to CONNEC messages is `mqtt.msgtype == 1`. Since we are interested in the mqttv3.1 protocol, we apply `mqtt.ver == 3`, where 3 corresponds to the version of interest. Similar to Q8 we open Statistics->I/O Graph. There in the "Display Filter" field we insert our filter, change the Y axis to AVG, inserted `mqtt.len` in the Y Field, increase the interval and observe, that the average message length is approximately equal to 63.59.

In order to explain the variance in the msg length, we should take a look at the structure of the packet.



We can see that there are fixed parts as well as the variable length parts inside. The maximum length of a packet in this scenario is defined by the broker. If we take a look at the packet with the smallest(right) and the largest(left) length among the given ones, we can more information specified inside the larger one, indicating that there are optional fields in the mqttv3.1 messages (ex. user name, password, will message and etc. Their presence depends on the presence of the corresponding flags in the message). Below illustrated the package with the smallest and with the largest lengths.

Wireshark · Packet 5181 · homework1.pcapng

```

Frame 5181: 94 bytes on wire (752 bits), 94 bytes captured (752 bits) on interface
Linux cooked capture
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 137.135.83.217
Transmission Control Protocol, Src Port: 51743, Dst Port: 1883, Seq: 1, Ack: 1,
MQ Telemetry Transport Protocol, Connect Command
  Header Flags: 0x10, Message Type: Connect Command
  Msg Len: 36
  Protocol Name Length: 6
  Protocol Name: MQTSDP
  Version: MQTT v3.1 (3)
  Connect Flags: 0x02, QoS Level: At most once delivery (Fire and Forget), Clean
    0... .. = User Name Flag: Not set
    .0... .. = Password Flag: Not set
    ..0... .. = Will Retain: Not set
    ...0... .. = QoS Level: At most once delivery (Fire and Forget) (0)
    ....0... .. = Will Flag: Not set
    .....1... .. = Clean Session Flag: Set
    .....0... .. = (Reserved): Not set
  Keep Alive: 490
  Client ID Length: 22
  Client ID: 4QZxNCWkNBp0uypGckMC6h

```

Wireshark · Packet 3619 · homework1.pcapng

```

Frame 3619: 191 bytes on wire (1528 bits), 191 bytes captured (1528 bits) on
Linux cooked capture
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 59119, Dst Port: 1883, Seq: 1, Ac
MQ Telemetry Transport Protocol, Connect Command
  Header Flags: 0xd6, User Name Flag, Password Flag, QoS Level: Exactly
  Msg Len: 121
  Protocol Name Length: 6
  Protocol Name: MQTSDP
  Version: MQTT v3.1 (3)
  Connect Flags: 0xd6, User Name Flag, Password Flag, QoS Level: Exactly
    1... .. = User Name Flag: Set
    .1... .. = Password Flag: Set
    ..0... .. = Will Retain: Not set
    ...1... .. = QoS Level: Exactly once delivery (Assured Delivery) (2)
    ....1... .. = Will Flag: Set
    .....1... .. = Clean Session Flag: Set
    .....0... .. = (Reserved): Not set
  Keep Alive: 442
  Client ID Length: 22
  Client ID: 7yvFP1YJq2W4AasXV0osml
  Will Topic Length: 36
  Will Topic: factory/department2/section2/deposit
  Will Message Length: 15
  Will Message: error: tvmgvjml
  User Name Length: 13
  User Name: administrator
  Password Length: 13
  Password: administrator

```